Internship Progress Dashboard API Documentation

1. Introduction

This document provides a comprehensive guide to the RESTful API for the **Internship Progress Dashboard**, a full-stack application designed to manage, track, and report on student internship progress. The system facilitates task assignment by mentors (Admins) and progress tracking by students (Users), utilizing real-time updates and robust role-based access control.

What the API Does

The API serves as the backbone for the Internship Progress Dashboard, enabling secure user authentication, managing tasks and submissions, providing a real-time progress dashboard, and delivering reports and analytics to mentors.

Base URL

All requests should be prefixed with the base URL:/api/v1

(Note: In a local development environment, this might be `http://localhost:8000/api/v1`)

API Version

This documentation describes **Version 1 (v1)** of the API.

2. Authentication

The API uses **JSON Web Tokens (JWT)** for secure, stateless authentication. Access to most endpoints requires a valid access token.

JWT Login Flow

1. A user (Student or Admin) sends credentials (`email` and `password`) to the `/auth/login` endpoint.
2. Upon successful validation, the API returns an HTTP `200 OK` response containing a JWT access token.
3. The client stores this token securely.

4. For subsequent protected requests, the client must include this token in the request header.

Token Format

The token is a standard compact JWT format (Base64Url encoded segments separated by dots: `header.payload.signature`). The payload contains necessary claims, including the user ID and the assigned `role` (`student` or `admin`).

How to Pass Token in Headers

The JWT must be passed in the `Authorization` header using the `Bearer` scheme:

Header Key	Value Format	Example
Authorization	Bearer	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

If the token is missing, expired, or invalid, the API will return an HTTP `401 Unauthorized` status code.

3. Roles & Permissions

The system employs **Role-Based Access Control (RBAC)** to ensure users only access resources relevant to their designation.

Student Role (`student`)

Students are the primary users of the system.
Feature	Access Level
Authentication	Register, Login, View Profile
Task Management	View assigned tasks, Submit work
Progress	View personal progress dashboard
Notifications	View and mark notifications as read
Admin/Reports APIs	**Forbidden**

Admin Role (`admin`)

The Admin role is combined with the Mentor responsibilities. This role has the highest level of access.
Feature	Access Level

Authentication	Register, Login, View Profile
Task Management	Create, View All, Update, Delete tasks, Review submissions
User/Student Management	View all users, Delete users, View specific student profiles
Reports	Full access to reports and analytics

Access Rules

All endpoints are protected and enforce permission checks based on the role embedded in the JWT payload.

4. API Endpoints (Focus: ADMIN)

This section details all available API endpoints, categorized by functionality.

🔐 Auth APIs

Endpoint Name	HTTP Method	URL	Description
User Registration	POST	`/auth/register`	Creates a new user account (Student by default).
User Login	POST	`/auth/login`	Authenticates a user and issues a JWT.
User Profile	GET	`/auth/profile`	Retrieves the authenticated user's details. Requires valid JWT.

POST /auth/register

- **Description:** Creates a new user (student) account.
- **Request Body (JSON):**

```
{
  "full_name": "<span type="placeholder" placeholder-type="person"></span>",
  "email": "student@example.com",
  "password": "strongpassword123",
  "role": "student"
}
```

- **Response (201 Created):**


```
{
  "user_id": "60c72b2f9c1e712a4c8d5f30",
  "email": "student@example.com",
  "role": "student",
  "message": "User registered successfully."
}
```


- **Status Codes:** 201 Created, 400 Bad Request (Validation Error), 409 Conflict (Email already exists)

🛠️ Admin (Mentor Combined) APIs

These endpoints require the `admin` role.

Task Management

Endpoint Name	HTTP Method	URL	Description
Create Task	POST	`/tasks/create`	Creates a new task and assigns it to students.
Update Task	PUT	`/tasks/update/{id}`	Updates details of an existing task.
Delete Task	DELETE	`/tasks/{id}`	Permanently deletes a task.
View All Tasks	GET	`/tasks`	Retrieves a list of all tasks.

POST /tasks/create

- **Description:** Creates a new task.
- **Request Body (JSON):**


```
{
  "title": "Setup Development Environment",
  "description": "Install Python, FastAPI, MongoDB, and required tools.",
```

```
  "assigned_to": ["60c72b2f9c1e712a4c8d5f30", "60c72b2f9c1e712a4c8d5f31"],
  "due_date": "<span type="placeholder" placeholder-type="date"></span>",
  "max_score": 100
}
```


- **Response (201 Created):**


```
{
  "task_id": "60c72c1c9c1e712a4c8d5f32",
  "message": "Task created and assigned successfully."
}
```


PUT /tasks/update/{id}

- **Description:** Updates task details. `{id}` is the task ID.
- **Request Body (JSON):** Accepts partial updates (e.g., only updating `due_date`).
- **Status Codes:** 200 OK, 404 Not Found, 403 Forbidden

Submission Review

Endpoint Name	HTTP Method	URL	Description
Review Submission	PUT	`/tasks/review`	Reviews a student's task submission and assigns a score.

PUT /tasks/review

- **Description:** Reviews a specific task submission from a student.
- **Request Body (JSON):**


```
{
  "submission_id": "60c72d009c1e712a4c8d5f33",
  "score": 85,
  "feedback": "Excellent work on configuration. Consider better variable naming.",
  "status": "completed"
```

}

- **Response (200 OK):**

{

"submission_id": "60c72d009c1e712a4c8d5f33",

"message": "Submission reviewed and progress updated."

}

Student Management

Endpoint Name	HTTP Method	URL	Description
View All Students	GET	`/students`	Retrieves a list of all users with the `student` role.
View Student Details	GET	`/students/{id}`	Retrieves detailed profile and progress for a specific student.

User Management

Endpoint Name	HTTP Method	URL	Description
View All Users	GET	`/users`	Retrieves a list of all users (Admins and Students).
Delete User	DELETE	`/users/{id}`	Deletes a user account and associated data.

Reports & Analytics

Endpoint Name	HTTP Method	URL	Description
Generate Reports	GET	`/reports`	Generates high-level progress reports (e.g., submission rates, average scores).
View Analytics	GET	`/analytics`	Retrieves detailed, filterable data for deep analysis.

- **GET /reports Response (200 OK):**

{

"total_students": 50,

"completion_rate": "75%",

"average_score": 88.5,

"top_students": [

{"name": "",
"score": 95}

]

}

👨‍🎓 Student APIs

These endpoints are accessible to users with the `student` role.

Endpoint Name	HTTP Method	URL	Description
View Assigned Tasks	GET	`/tasks`	Retrieves a list of tasks assigned to the authenticated student.
Submit Task	POST	`/tasks/submit`	Submits work for a specific task.
View Personal Progress	GET	`/progress`	Retrieves the authenticated student's overall progress summary.

POST /tasks/submit

- **Description:** Submits a task for review.
- **Request Body (JSON):**

{

"task_id": "60c72c1c9c1e712a4c8d5f32",

"submission_url": "https://github.com/student/repo/task1",

"comments": "Completed task. Ready for review."

}

- **Response (201 Created):**

{

"submission_id": "60c72d009c1e712a4c8d5f33",

"message": "Task submitted successfully. Awaiting review."

}

📊 Dashboard APIs

These endpoints support the dynamic display of the dashboard. Both roles can access their respective dashboard data.

Endpoint Name	HTTP Method	URL	Description
Dashboard Summary	GET	`/dashboard/summary`	Retrieves key metrics (e.g., Tasks due this week, pending reviews).
Dashboard Activity	GET	`/dashboard/activity`	Retrieves a feed of recent activities (e.g., new task assigned, submission reviewed).

🔔 Notifications

These endpoints handle real-time and persistent notifications for both user roles.

Endpoint Name	HTTP Method	URL	Description
Get Notifications	GET	`/notifications`	Retrieves the authenticated user's unread notifications.
Mark as Read	PUT	`/notifications/read`	Marks one or all notifications as read.

PUT /notifications/read

- **Description:** Marks notifications as read.

- **Request Body (JSON):**
  - To mark a specific notification: `{"notification_id": "60c72d009c1e712a4c8d5f34"}`
  - To mark all unread notifications: `{"mark_all": true}`
- **Status Codes:** 200 OK

5. Data Models (Schemas)

The following schemas define the core data structures used in the API. All MongoDB IDs are represented as 24-character hexadecimal strings.

User

Represents both Student and Admin accounts.

Field	Type	Description
`id`	string (ObjectID)	Unique user identifier.
`full_name`	string	User's full name.
`email`	string	Unique email address (used for login).
`role`	enum (`student`, `admin`)	User's role and permission level.
`is_active`	boolean	Account status.
`created_at`	datetime	Timestamp of creation.

Example JSON:{

"id": "60c72b2f9c1e712a4c8d5f30",

"full_name": "",

"email": "mentor@api.com",

"role": "admin",

"is_active": true,

"created_at": ""

}

Task

Represents an assignment given by an Admin/Mentor.

Field	Type	Description
`id`	string (ObjectID)	Unique task identifier.
`title`	string	Short name for the task.
`description`	string	Detailed task instructions.
`assigned_by`	string (ObjectID)	Admin ID who created the task.
`assigned_to`	array of string (ObjectID)	List of Student IDs assigned.
`due_date`	datetime	Submission deadline.
`max_score`	integer	Maximum score obtainable for the task.
`status`	enum (`pending`, `completed`, `overdue`)	Current task status.

Example JSON:{

"id": "60c72c1c9c1e712a4c8d5f32",

"title": "FastAPI Integration Test",

"description": "Write unit and integration tests for the authentication module.",

"assigned_by": "60c72b2f9c1e712a4c8d5f30",

"assigned_to": ["60c72b2f9c1e712a4c8d5f31"],

"due_date": "",

"max_score": 100,

"status": "pending"

}

Submission

Represents a Student's attempt to complete a task